

Bayesian modelling

Monte Carlo methods, Markov chains and
the Metropolis–Hastings algorithm

Léo Belzile

Last compiled Tuesday Sep 15, 2026

Bayesian inference beyond conjugate models

How to circumvent the problem of intractable posteriors?

- simulation-based methods: accept-reject, Markov chain Monte Carlo, particle filters, etc.
- deterministic methods: (integrated nested) Laplace approximations, variational Bayes, expectation propagation, etc.

We focus on Monte Carlo methods.

Simulation algorithms: inversion method

If F is an absolutely continuous distribution function, then

$$F(X) \sim \text{unif}(0, 1).$$

The inversion method consists in applying the quantile function F^{-1} to $U \sim \text{unif}(0, 1)$, viz.

$$F^{-1}(U) \sim X.$$

Inversion method for truncated distributions

Consider a random variable Y with distribution function F .

If X follows the same distribution as Y , but restricted over the interval $[a, b]$, then

$$\Pr(X \leq x) = \frac{F(x) - F(a)}{F(b) - F(a)}, \quad a \leq x \leq b,$$

Therefore,

$$F^{-1}[F(a) + \{F(b) - F(a)\}U] \sim X.$$

Fundamental theorem of simulation

Consider a d -variate random vector $\mathbf{X}$, independent of $U \sim \text{unif}(0, 1)$ and $c > 0$. If $(\mathbf{X}, U)$ is uniformly distributed on the set

$$\mathcal{A}_f = \{(\mathbf{x}, u) : 0 \leq u \leq cf(\mathbf{x})\},$$

then $\mathbf{X}$ has density $f(\mathbf{x})$.

- f is the marginal density of $\mathbf{X}$ since $f(\mathbf{x}) = \int_0^{f(\mathbf{x})} du$.
- If we can simulate uniformly from $\mathcal{A}_f$, then, we can discard the auxiliary variable u . See Devroye (1986), Theorem 3.1.

Fundamental theorem of simulation in picture

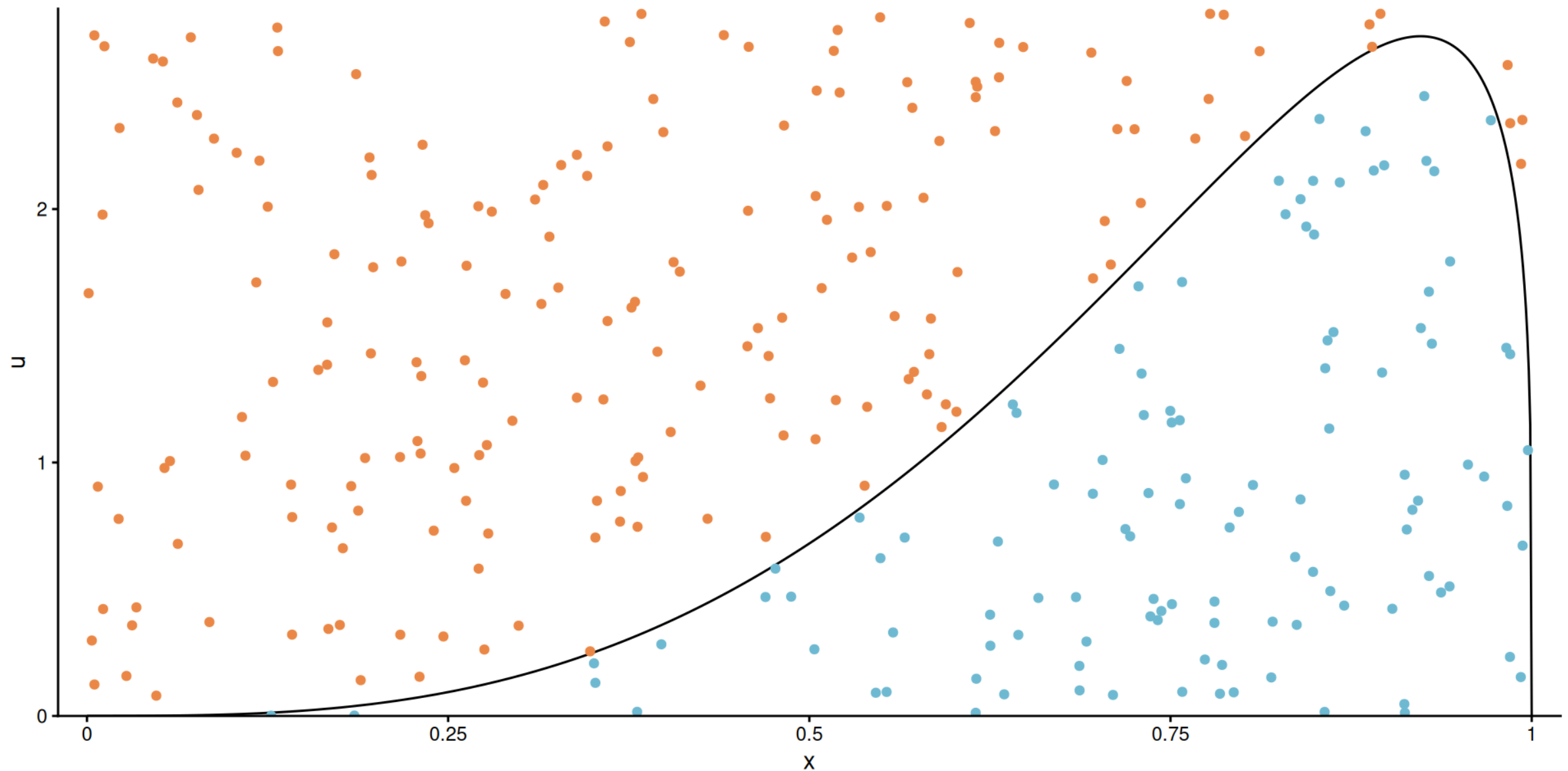


Figure 1: Illustration of the fundamental theorem of simulation. All points in blue below the density curve belong to $\mathcal{A}_f$.

Simulation algorithms: accept-reject

- **Target:** sample from density $p(x)$ (hard to sample from)
- **Proposal:** find a density $q(x)$ with nested support, $\text{supp}(p) \subseteq \text{supp}(q)$, such that for all $x \in \text{supp}(p)$,

$$\frac{p(x)}{q(x)} \leq C, \quad C \geq 1.$$

Uses the fundamental theorem of simulation by finding an envelope $Cq(x)$ that is over the density $p(x)$.

Rejection sampling algorithm

1. Generate X from proposal with density $q(x)$.
2. Compute the ratio $R \leftarrow p(X)/q(X)$.
3. If $CU \leq R$ for $U \sim \text{unif}(0, 1)$, return X , else go back to step 1.

Accept-reject illustration

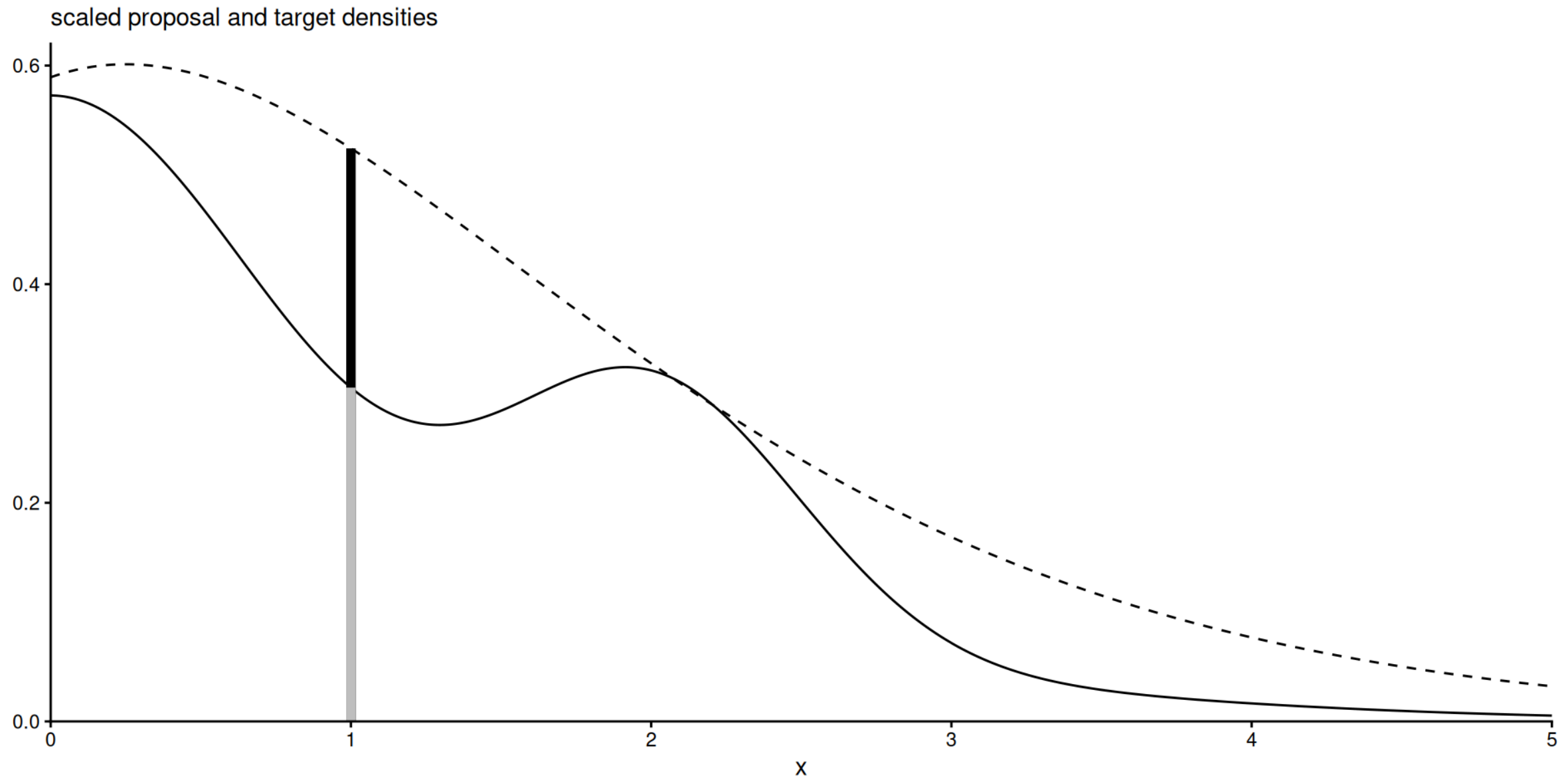


Figure 2: Target density (full) and scaled proposal density (dashed): the vertical segment at $x = 1$ shows the percentage of acceptance for a uniform slice under the scaled proposal, giving an acceptance ratio of 0.58.

Remarks on rejection sampling

- Acceptance rate is $1/C$:
 - we need on average C draws from q to get one from p .
- q must be more heavy-tailed than p
 - e.g., $q(x)$ Student- t for $p(x)$ Gaussian.
- q should be cheap and easy to sample from!

Designing a good proposal density

Good choices must satisfy the following constraints:

- pick a family $q(x)$ so that

$$C = \operatorname{argmax}_x \frac{p(x)}{q(x)}$$

is as close to 1 as possible.

- you can use numerical optimization with target

$$f(x) = \log p(x) - \log q(x)$$

to find the mode x^* and the upper bound $C = \exp f(x^*)$.

Truncated Gaussian via accept-reject

Consider sampling $Y \sim \text{Gauss}(\mu, \sigma^2)$, but truncated in the interval (a, b) . The target density is

$$p(x; \mu, \sigma, a, b) = \frac{1}{\sigma} \frac{\phi\left(\frac{x-\mu}{\sigma}\right)}{\Phi(\beta) - \Phi(\alpha)}.$$

for $\alpha = (a - \mu)/\sigma$ and $\beta = (b - \mu)/\sigma$. where $\phi(\cdot)$, $\Phi(\cdot)$ are respectively the density and distribution function of the standard Gaussian distribution.

Accept-reject (crude version)

1. Simulate $X \sim \text{Gauss}(\mu, \sigma^2)$
2. reject any draw if $X < a$ or $X > b$.

The acceptance rate is $C^{-1} = \{\Phi(\beta) - \Phi(\alpha)\}$

```
1 # Standard Gaussian truncated on [0,1]
2 candidate <- rnorm(1e5)
3 trunc_samp <- candidate[candidate >= 0 & candidate <= 1]
4 # Acceptance rate
5 length(trunc_samp)/1e5
```

```
[1] 0.34289
```

```
1 # Theoretical acceptance rate
2 pnorm(1)-pnorm(0)
```

```
[1] 0.3413447
```

Accept-reject for truncated Gaussian

Since the Gaussian is a location scale family, the inversion method gives

$$X \sim \mu + \sigma \Phi^{-1} [\Phi(\alpha) + \{\Phi(\beta) - \Phi(\alpha)\}U]$$

We however need to evaluate Φ numerically (no closed-form expression).

The method fails for *rare event* simulation because the computer returns

- $\Phi(x) = 0$ for $x \leq -39$
- $\Phi(x) = 1$ for $x \geq 8.3$,

implying that $a \leq 8.3$ for this approach to work (Botev & L'Écuyer, 2017).

Simulating tails of Gaussian variables

We consider simulation from a standard Gaussian truncated above $a > 0$

Write the density of the truncated Gaussian as ([Devroye, 1986, p. 381](#))

$$f(x) = \frac{\exp(-x^2/2)}{\int_a^\infty \exp(-z^2/2)dz} = \frac{\exp(-x^2/2)}{c_1}.$$

Note that, for $x \geq a$,

$$c_1 f(x) \leq \frac{x}{a} \exp\left(-\frac{x^2}{2}\right) = a^{-1} \exp\left(-\frac{a^2}{2}\right) g(x);$$

where $g(x)$ is the density of a Rayleigh variable shifted by a .

The constant $C = \exp(-a^2/2)(c_1 a)^{-1}$ approaches 1 quickly as $a \rightarrow \infty$ (asymptotically optimality).

Accept-reject: truncated Gaussian with Rayleigh

The shifted Rayleigh has distribution function

$$G(x) = 1 - \exp\{(a^2 - x^2)/2\}, x \geq a.$$

! Marsaglia algorithm

1. Generate a shifted Rayleigh above a , $X \leftarrow \{a^2 - 2 \log(U)\}^{1/2}$ for $U \sim \text{unif}(0, 1)$
2. Accept X if $XV \leq a$, where $V \sim \text{unif}(0, 1)$.

For sampling on $[a, b]$ with a very large, propose from a Rayleigh truncated above at b (Botev & L'Écuyer, 2017).

```
1 a <- 8.3
2 niter <- 1000L
3 X <- sqrt(a^2 + 2*rexp(niter))
4 samp <- X[runif(niter)*X <= a]
```

Monte Carlo integration

Calculate $\mathbf{E}_X\{g(X)\}$ of X with density function p on $\mathcal{X}$.

Monte Carlo integration proceeds by drawing B independent samples $x_1, \dots, x_B$ from p on $\mathcal{X}$ and evaluating the empirical average of g ,

$$\mathbf{E}\{g(X)\} = \int_{\mathcal{X}} g(x)p(x)\mathrm{d}x \approx \hat{\mathbf{E}}\{g(X)\} = \frac{1}{B} \sum_{b=1}^B g(x_b).$$

Importance sampling

We can calculate the integral $\mathbf{E}\{g(X)\} = \int_{\mathcal{X}} g(x)p(x)\mathrm{d}x$ by considering instead draws from a density $q(\cdot)$ if $\mathcal{X} \subseteq \text{supp}(q)$. Then,

$$\mathbf{E}\{g(X)\} = \int_{\mathcal{X}} g(x) \frac{p(x)}{q(x)} q(x) \mathrm{d}x.$$

Monte Carlo estimator for importance sampling

An alternative Monte Carlo estimator, which is biased but has lower variance, is obtained by drawing independent $x_1, \dots, x_B$ from q and taking instead the weighted average of

$$\tilde{\mathbb{E}}\{g(X)\} = \frac{B^{-1} \sum_{b=1}^B w_b g(x_b)}{B^{-1} \sum_{b=1}^B w_b}.$$

with weights $w_b = p(x_b)/q(x_b)$. The latter equal 1 on average, so one could omit the denominator without harm.

Importance sampling for the variance of a beta distribution

Consider $X \sim \text{beta}(\alpha, \alpha)$ for $\alpha > 1$, with expectation $\mathbf{E}(X) = 0.5$ since the density is symmetric.

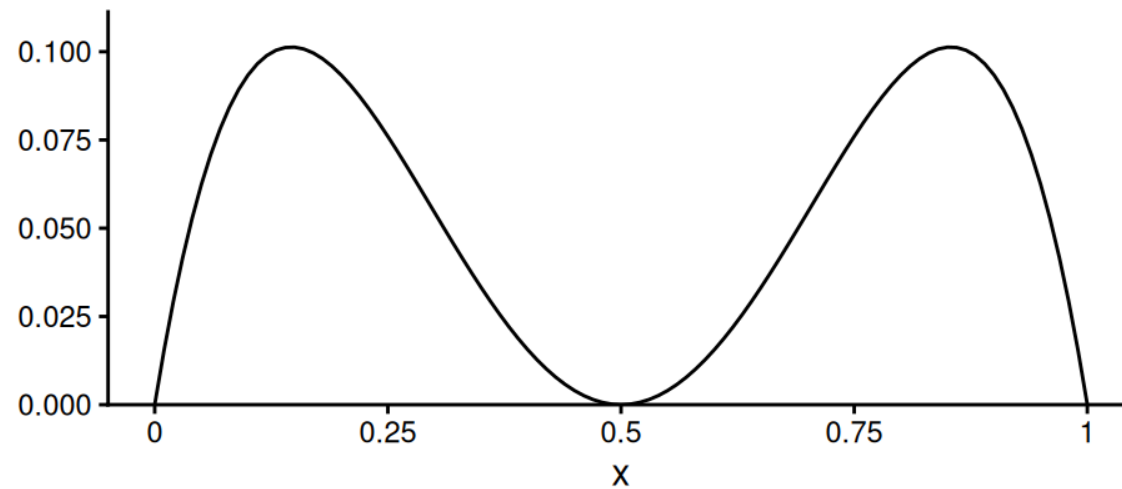
The variance is

$$\text{Va}(X) = \mathbf{E}\{(X - 0.5)^2\} = \frac{1}{4 \cdot (2\alpha + 1)}.$$

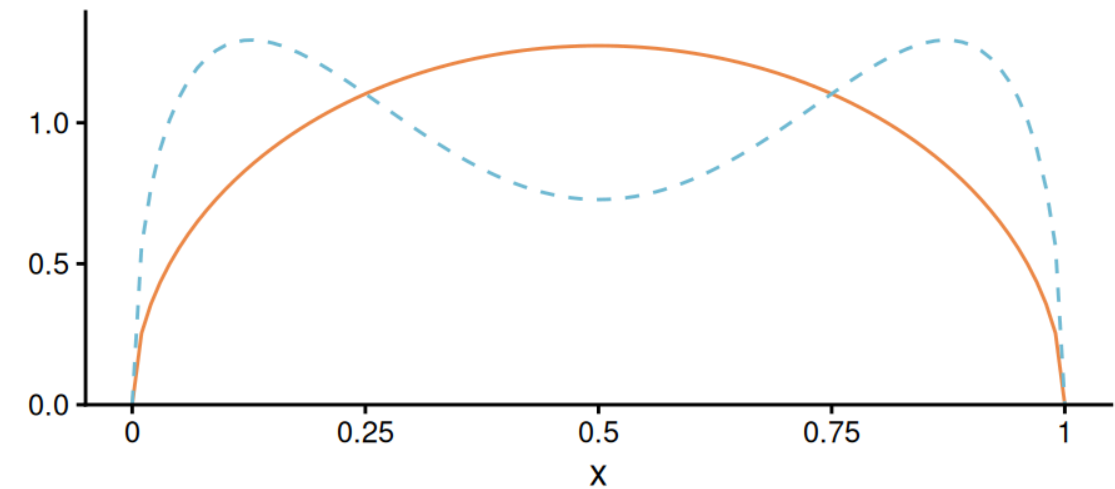
We consider an equiweighted (bimodal) mixture of $\text{beta}(\alpha, 3\alpha)$ and $\text{beta}(3\alpha, \alpha)$ as importance sampling density.

Importance sampling for variance of a beta distribution

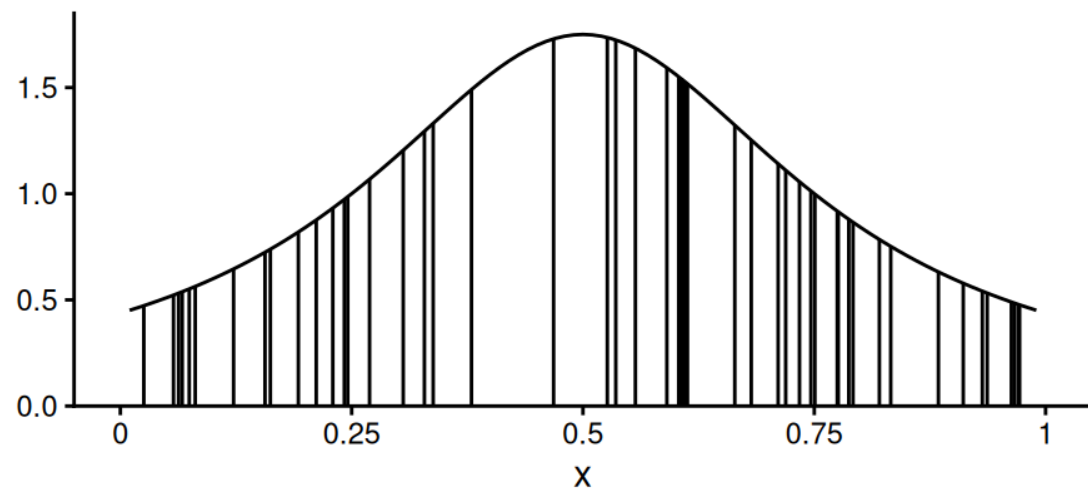
$$f(x) = (x - 0.5)^2 p(x)$$



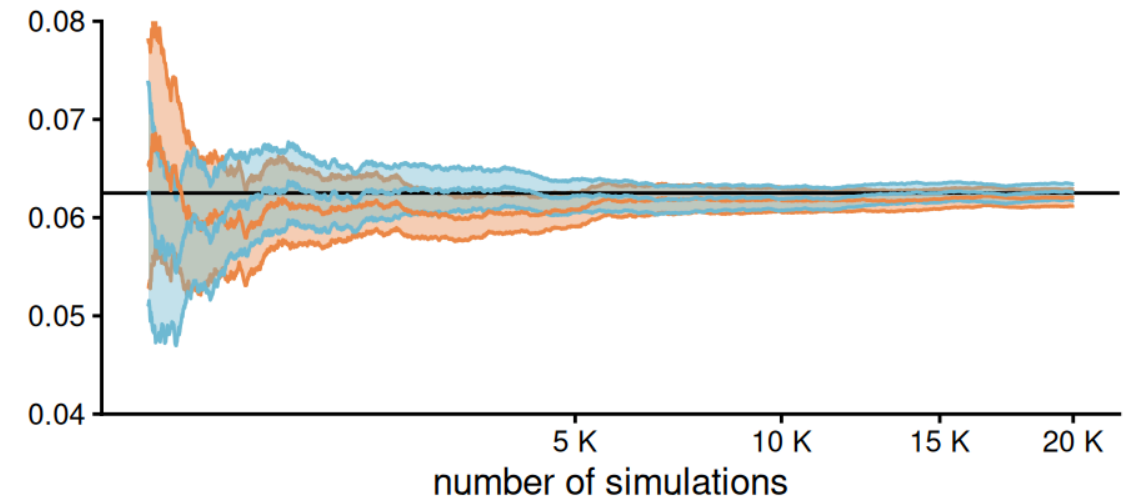
density of Monte Carlo generator



weights



Variance estimation



Code for importance sampling of beta variance

```

1 B <- 2e4L; alpha <- 1.5; factor <- 3
2 # Mode at the mean 0.5
3 X0 <- rbeta(n = B, alpha, alpha)
4 px <- function(x){dbeta(x, alpha, alpha)}
5 # Importance sampling density - mixture of two betas (alpha, factor*alpha)
6 X1 <- ifelse(
7   runif(B) < 0.5,
8   rbeta(B, alpha, factor*alpha),
9   rbeta(B, factor*alpha, alpha))
10 qx <- function(x){
11   0.5*dbeta(x, alpha, factor*alpha) + 0.5*dbeta(x, factor*alpha, alpha)}
12 # Function to integrate - gives variance of a symmetric beta distribution
13 g <- function(x){(x - 0.5)^2}
14 # Weights for importance sampling
15 w <- px(X1)/qx(X1)

```

Code for importance sampling of beta variance (continued)

```

1 # Monte Carlo integration
2 mc_est <- mean(g(X0))
3 mc_var <- var(g(X0))/B
4 # Importance sampling weighted mean and variance
5 is_est <- weighted.mean(g(X1), w = w) # equivalent to mean(g(X1)*w)/mean(w)
6 is_var <- sum(w^2*(g(X1) - is_est)^2)/ (sum(w)^2)
7 # True value for the beta variance
8 th_est <- 1/(4*(2*alpha+1))
9 # Point estimates and differences
10 round(c(true = th_est, "monte carlo" = mc_est, "importance sampling" = is_est),4)

```

true	monte carlo	importance sampling
0.0625	0.0629	0.0627

```

1 # Ratio of std. errors for means
2 mc_var/is_var # value > 1 means that IS is more efficient

```

```
[1] 1.107183
```

Markov chains

Plain ordinary Monte Carlo is great, but few algorithms are generic enough to be useful in complex high-dimensional problems.

We will instead typically build Markov chains that target an invariant stationary distribution.

Caveats?

Markov chain Monte Carlo methods generate **correlated** draws.

Questions:

1. can we use them as ordinary independent samples?
2. what is the price to pay?

We need to do a little theoretical detour to answer these questions.

Stationarity and Markov property

A stochastic process is

- **(strongly) stationary** if the distribution of $\{X_1, \dots, X_t\}$ is the same as that of $\{X_{n+1}, \dots, X_{t+n}\}$ for any value of n and given t .
- **weakly stationary** if $E(X_t) = \mu$ for all t , and $\text{Cov}(X_t, X_{t+h}) = \gamma_h$ does not depend on t .
- **Markov** if it satisfies the Markov property: given the current state of the chain, the future only depends on the current state and not on the past.

Strong stationarity implies weak stationarity.

Autoregressive process of order 1

Consider a first-order autoregressive process, or AR(1),

$$Y_t = \mu + \phi(Y_{t-1} - \mu) + \varepsilon_t,$$

where

- ϕ is the lag-one correlation,
- μ the global mean
- ε_t is an iid innovation with mean zero and variance σ^2

If $|\phi| < 1$, the process is stationary, otherwise the variance increases with t .

Unconditional moments via tower law

If the process is weakly stationary, then $\mathbf{E}_{Y_t}(Y_t) = \mathbf{E}_{Y_{t-1}}(Y_{t-1})$

$$\begin{aligned}\mathbf{E}_{Y_t}(Y_t) &= \mathbf{E}_{Y_{t-1}} \left\{ \mathbf{E}_{Y_t|Y_{t-1}}(Y_t) \right\} \\ &= \mu(1 - \phi) + \phi \mathbf{E}_{Y_{t-1}}(Y_{t-1})\end{aligned}$$

and so the unconditional mean is μ . For the variance, we have

$$\begin{aligned}\mathbf{Va}_{Y_t}(Y_t) &= \mathbf{E}_{Y_{t-1}} \left\{ \mathbf{Va}_{Y_t|Y_{t-1}}(Y_t) \right\} + \mathbf{Va}_{Y_{t-1}} \left\{ \mathbf{E}_{Y_t|Y_{t-1}}(Y_t) \right\} \\ &= \sigma^2 + \mathbf{Va}_{Y_{t-1}} \left\{ \mu + \phi(Y_{t-1} - \mu) \right\} \\ &= \sigma^2 + \phi^2 \mathbf{Va}_{Y_{t-1}}(Y_{t-1}).\end{aligned}$$

and the unconditional variance is $\mathbf{Va}_{Y_t}(Y_t) = \sigma^2 / (1 - \phi^2)$.

Autocovariance and Markov property

The covariance at lag k , in terms of innovations, gives

$$\gamma_k = \mathbf{Co}(Y_t, Y_{t-k}) = \mathbf{Va}(\phi Y_{t-1}, Y_{t-k}) + \mathbf{Va}(\varepsilon_t, Y_{t-k}) = \phi \gamma_{k-1}$$

since ε_t is independent of the past. We thus find

$$\gamma_k = \phi^k \mathbf{Va}(Y_t).$$

The AR(1) process is first-order Markov since the conditional distribution $p(Y_t \mid Y_{t-1}, \dots, Y_{t-p})$ equals $p(Y_t \mid Y_{t-1})$.

Variance of sample average

Intuitively, a sample of correlated observations carries less information than an independent sample of draws.

The variance of the sample average is

$$\begin{aligned}\text{Va} \left(\bar{Y}_T \right) &= \frac{1}{T^2} \sum_{t=1}^T \sum_{s=1}^T \text{Co}(Y_t, Y_s) \\ &= \frac{1}{T^2} \sum_{t=1}^T \text{Va}(Y_t) + \frac{2}{T^2} \sum_{t=1}^{T-1} \sum_{s=t+1}^T \text{Co}(Y_t, Y_s).\end{aligned}$$

Under independence and assuming stationarity, we get

$$\text{Va} \left(\bar{Y}_T \right) = \sigma^2 / T.$$

Variance of sample average, redux

If the second moments are finite, the scaled limiting variance of the sample mean simplifies to

$$\lim_{T \rightarrow \infty} T \text{Va} \left(\bar{Y}_T \right) = \tau^2 \left\{ 1 + 2 \sum_{t=1}^{\infty} \rho_t \right\}.$$

which is a function of

- the unconditional variance τ^2
- the lag- k autocorrelation, $\text{Cor}(Y_t, Y_{t+k}) = \rho_k$.

Effective sample size

The effective sample size is, loosely speaking, the equivalent number of observations if the B marginal posterior draws were independent and more formally

$$\text{ESS} = \frac{B}{\sum_{t=-\infty}^{\infty} \rho_t} = \frac{B}{\{1 + 2 \sum_{t=1}^{\infty} \rho_t\}}$$

where ρ_t is the lag t correlation, since $\rho_t = \rho_{-t}$ and $\rho_0 = 1$.

Effective sample size for AR(1)

The lag- k correlation of the stationary autoregressive process of order 1 is ϕ^k , so

$$1 + 2 \sum_{t=1}^{\infty} \rho_t = 1 + 2 \left(\frac{1}{1 - \phi} - 1 \right) = \frac{1 + \phi}{1 - \phi}.$$

whereas the unconditional variance of the AR(1) process is

$$\tau^2 = \frac{\sigma^2}{1 - \phi^2}.$$

Inefficiency curve for AR(1)

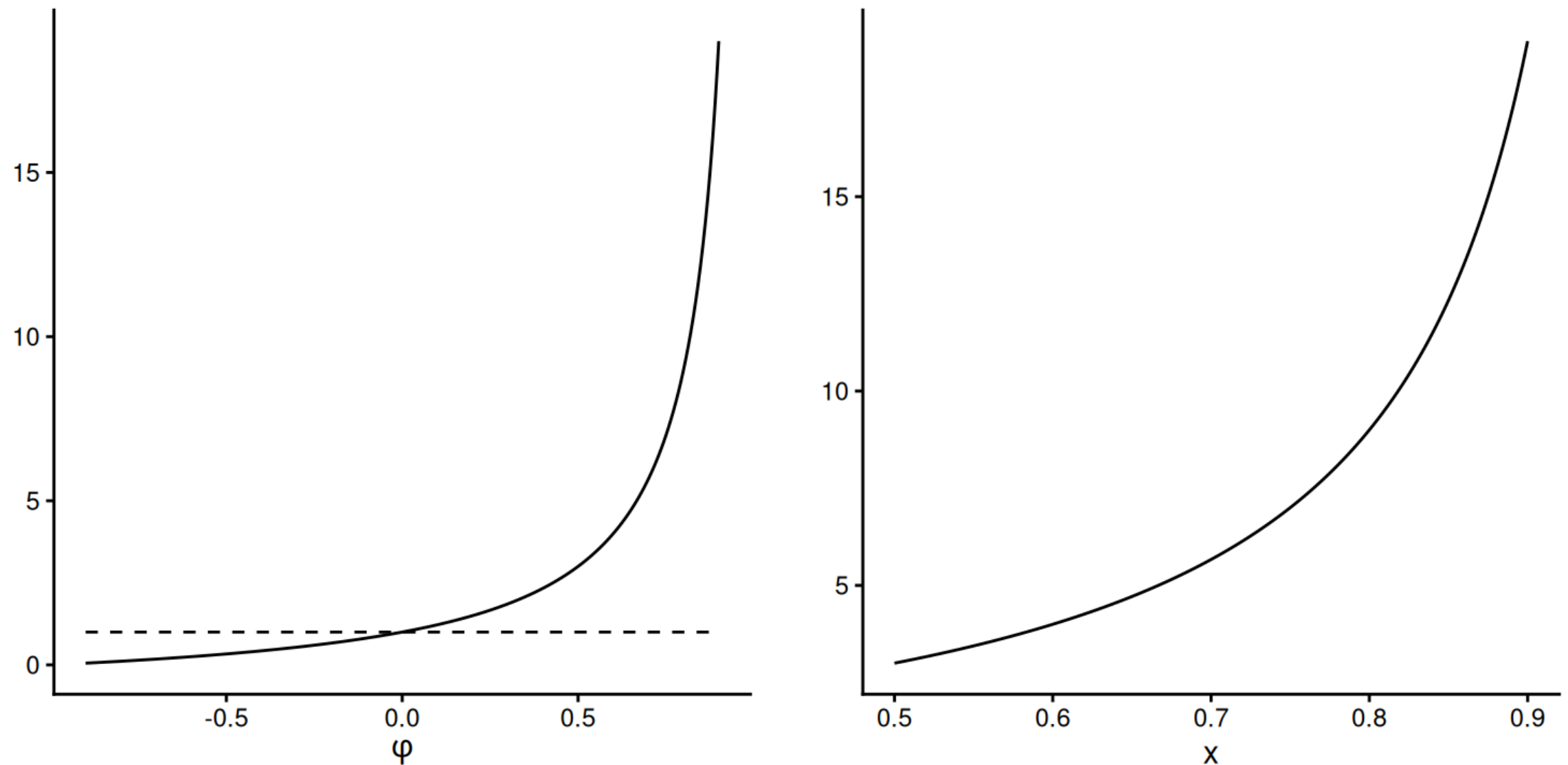


Figure 3: Left: scaled asymptotic variance of the sample mean for AR(1) (full line) and independent observations with unit marginal variance (dashed). Right: variance ratio for positive correlations for selected range.

To get the same precision for the mean of AR(1) process with $\phi \approx 0.75$ than with i.i.d. data, we would need 7 times as many observations.

Morale of the story

The price to pay for having correlated samples is

inefficiency

The higher the autocorrelation, the larger the variability of our estimators.

Standard error of mean for first-order autoregressive process

```
1 phi <- 0.8 # lag-one autocorrelation
2 sigma <- 2 # std. deviation of white noise
3 B <- 1e4L # chain length
4 x <- arima.sim( # generate AR(1)
5   model = list(ar = phi),
6   n = B,
7   rand.gen = rnorm,
8   sd = sigma
9 )
10 # Effective sample size / B
11 ess_factor_ar1 <- 1 / (-1 + 2 / (1 - phi))
12 # Unconditional variance tau_sq, obtained from var(x)
13 uvar_ar1 <- sigma^2 / (1 - phi^2)
14 # Theoretical std. error of sample mean
15 se_mean_ar1 <- sqrt(uvar_ar1 / B / ess_factor_ar1)
```

Standard errors estimation via nonparametric spectral analysis

Suppose that $X_1, \dots, X_B$ is a mean-stationary sequence.

Heidelberger & Welch (1981) recall that the variance of the sample mean is $S(0)/B$, where $S(\cdot)$ is the spectrum evaluated at zero and

$$S(0) = \sum_{k=-\infty}^{\infty} \gamma_k.$$

They propose to approximate $S(0)$ by

- calculating the raw periodogram (unsmoothed)
- fitting a gamma GLM to the log periodogram with a polynomial trend.

Standard errors estimation via parametric spectral analysis

Any continuous spectral density function can be approximated by an $AR(p)$ process (for p potentially infinitely large).

For an autoregressive process of order p , the spectral density of the $AR(p)$ is ([Percival & Walden, 2020](#), eq. 446b)

$$S(0) = \frac{\sigma^2}{\left(1 - \sum_{j=1}^p \phi_j\right)^2}.$$

This suggests

- Fitting an $AR(p)$ model (selecting p via AIC) using Yule–Walker or any other method.
- Calculating the spectral density at zero with the plug-ins $\hat{\phi}_1, \dots, \hat{\phi}_p$ and $\hat{\sigma}^2$.

Spectral estimation in R

```
1 # Heidelberg and Welch (1981)
2 coda::spectrum0(x)
```

\$spec

[1] 115.9407

```
1 # Code from coda::effectiveSize
2 ar_mod <- ar.yw(x)
3 suvar <- ar_mod$var.pred / (1 - sum(ar_mod$ar))^2
4 se3 <- sqrt(suvar / B) # variance of sample mean of AR(p)
5 ess3 <- var(x) / suvar * B
```

Standard errors for posterior means using batch means

Geyer (2011) recommends to segment the time series into batches

1. Break the chain of length B (after burn in) in K blocks of size $\approx K/B$.
2. Compute the sample mean of each segment.
3. Compute the standard deviation of the segments mean.
4. Rescale by $K^{-1/2}$ to get standard error of the global mean.

Illustration of batch means

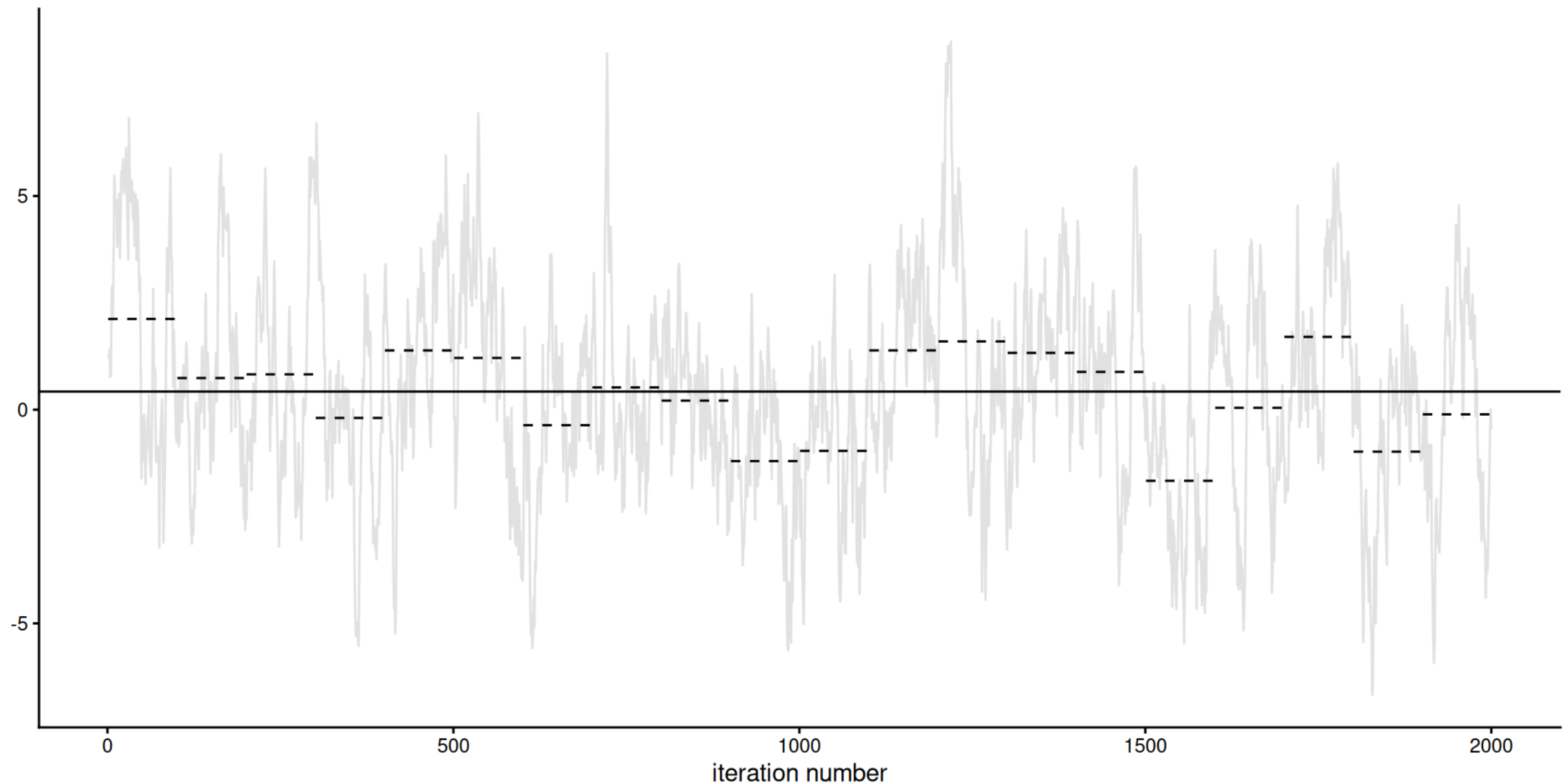


Figure 4: Calculation of the standard error of the posterior mean using the batch method.

Correlogram

We can look at the autocorrelation function to check the efficiency.

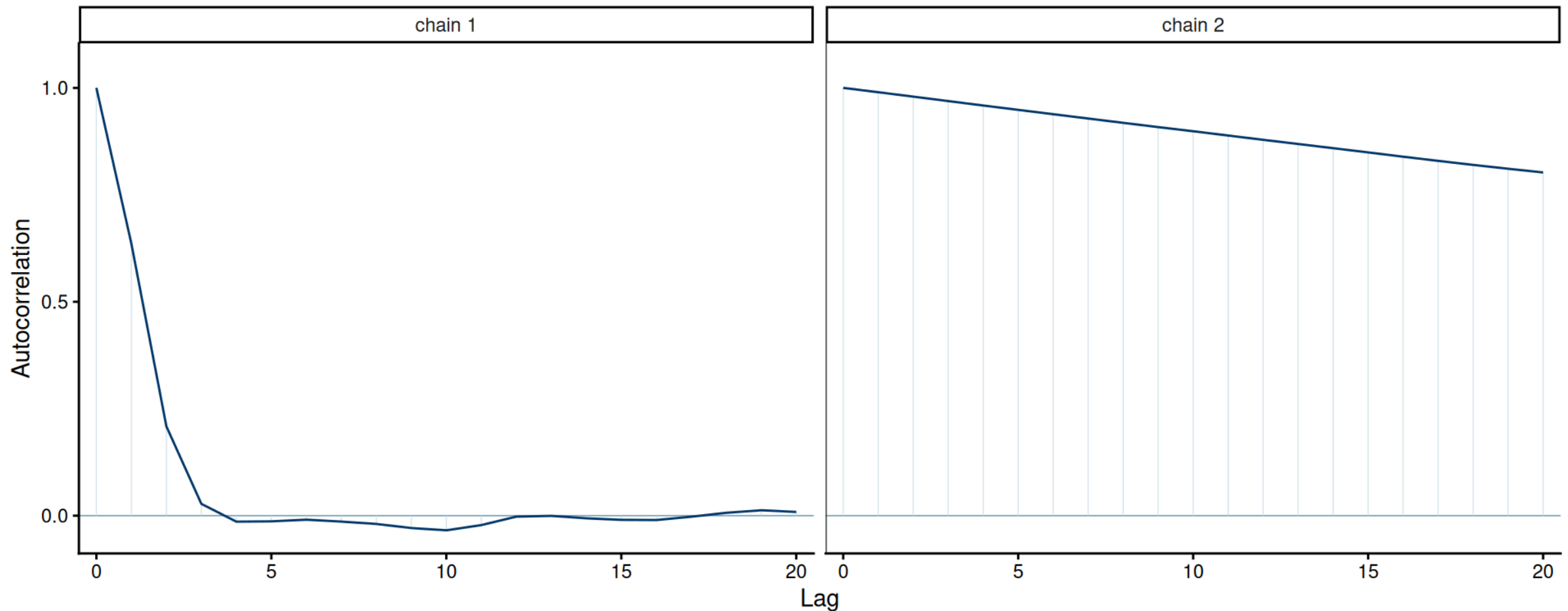


Figure 5: Correlogram of two Markov chains. These plots, often called acf or autocorrelation functions, show the lag-k sample autocorrelation against lag number.

Convergence of Markov chains

If a Markov chain is irreducible and acyclic, it has a unique stationary distribution.

- **irreducibility:** means that the chain can move from anywhere to anywhere, so it doesn't get stuck in part of the space forever.
- **acyclic:** cyclical chains loop around and visit periodically a state

Examples of cyclical or reducible chains

Consider discrete Markov chains over the integers 1, 2, 3 with transition matrices

$$P_1 = \begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0 & 0.4 & 0.6 \\ 0 & 0.5 & 0.5 \end{pmatrix}, \quad P_2 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

Chain 1 is reducible to $\{2, 3\}$, chain 2 is cyclical.

Law of large number (ergodic theorem)

Ergodicity means that two segments of a time series far enough apart act as independent.

Let $\{Y_t\}$ is a weakly stationary sequence with mean $E(Y_t) = \mu$ and $\gamma_h = \text{Cov}(Y_t, Y_{t+h})$. Then, if the autocovariance series is convergent, meaning

$$\sum_{h=0}^{\infty} |\gamma_h| < \infty,$$

then $\{Y_t\}$ is ergodic for the mean and $\bar{Y} \xrightarrow{p} \mu$.

Ergodicity and transformations

The ergodic theorem is a law of large numbers for stochastic processes that allows for serial dependence between observations, provided the latter is not too large.

Any transformation $g(\cdot)$ of a stationary and ergodic process $\{Y_t\}$ retains the properties, so $\bar{g} = T^{-1} \sum_{t=1}^T g(Y_t) \rightarrow \mathbf{E}\{g(Y_t)\}$ as $T \rightarrow \infty$.

The ergodic theorem holds even if the chain is cyclical.

Convergence and stationary distribution

Consider a transition P on $1, \dots, 5$ defined as

$$P = \begin{pmatrix} \frac{2}{3} & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{6} & \frac{2}{3} & \frac{1}{6} & 0 & 0 \\ 0 & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} & 0 \\ 0 & 0 & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \\ 0 & 0 & 0 & \frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

The stationary distribution is the value of the row vector $\mathbf{p}$, such that $\mathbf{p} = \mathbf{p}\mathbf{P}$ for transition matrix $\mathbf{P}$: we get $\mathbf{p} = (1, 2, 2, 2, 1)/8$.

Convergence of Markov chains

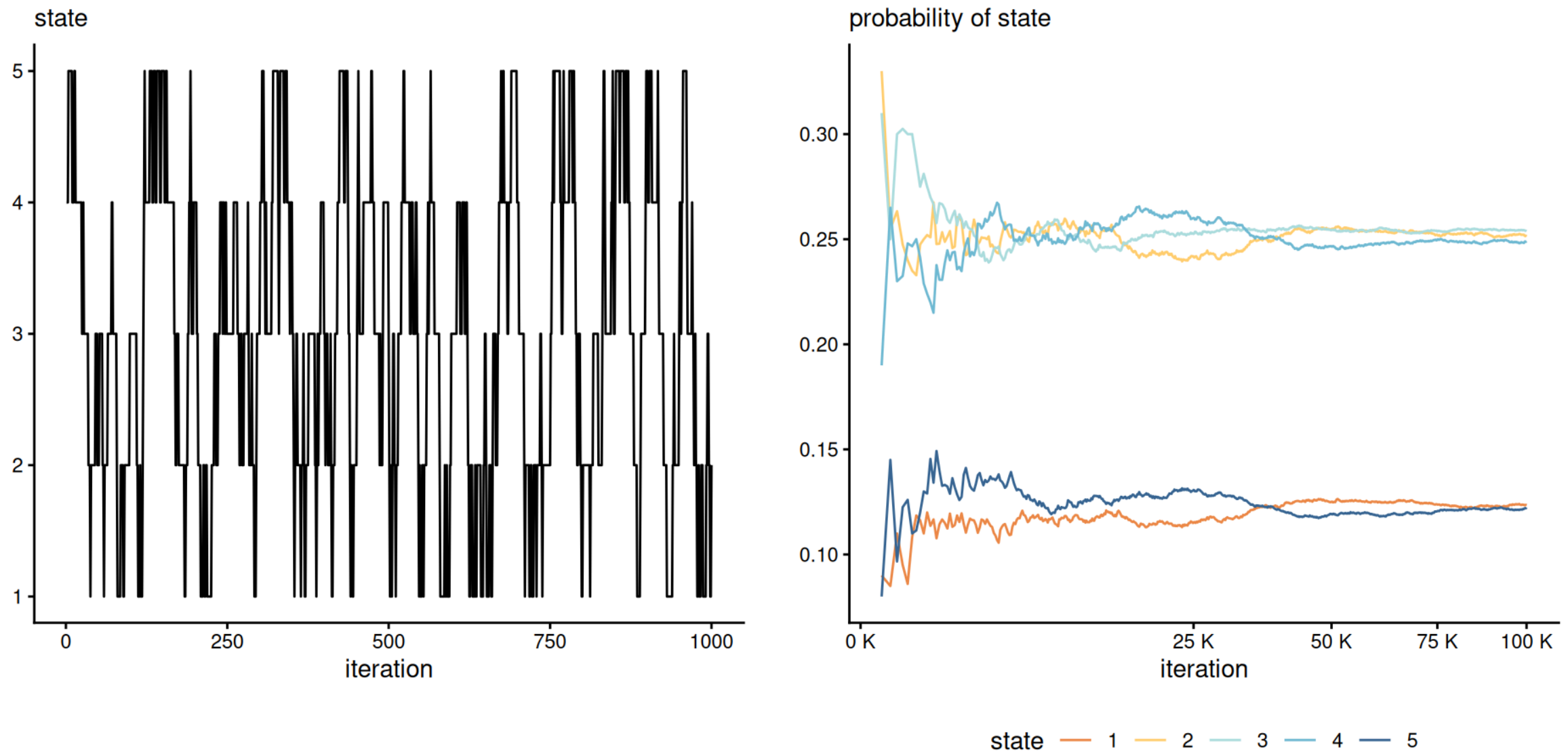


Figure 6: Discrete Markov chain on integers from 1 to 5, with traceplot of 1000 first iterations (left) and running mean plots of sample proportion of each state visited (right).

Markov chain Monte Carlo

We consider simulating from a distribution with associated density function proportional to $p(\cdot)$

- using an algorithm that generates a Markov chain ,
- without requiring knowledge of the normalizing factor.

We use a conditional density $q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{\text{cur}})$ to generate proposals from the current value $\boldsymbol{\theta}^{\text{cur}}$.

Metropolis–Hastings algorithm

Starting from an initial value $\boldsymbol{\theta}_0$: for $t = 1, \dots, T$

1. draw a proposal value $\boldsymbol{\theta}_t^* \sim q(\boldsymbol{\theta} \mid \boldsymbol{\theta}_{t-1})$.

2. Compute the acceptance ratio

$$R = \frac{p(\boldsymbol{\theta}_t^*)}{p(\boldsymbol{\theta}_{t-1})} \frac{q(\boldsymbol{\theta}_{t-1} \mid \boldsymbol{\theta}_t^*)}{q(\boldsymbol{\theta}_t^* \mid \boldsymbol{\theta}_{t-1})}$$

3. With probability $\alpha = \min\{R, 1\}$, accept the proposal and set $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_t^*$, otherwise set the value to the previous state, $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1}$.

Theory

The Metropolis–Hastings algorithm satisfies a technical condition (detailed balance) that ensures that the Markov chain generated is reversible.

- Thus, any draw from the posterior will generate a new realization from the posterior.
- Provided the starting value has non-zero probability under the posterior, the chain will converge to the stationarity distribution (albeit perhaps slowly).

Interpretation

- If $R > 1$, the proposal has higher density and we always accept the move.
- If we reject the move, the Markov chain stays at the current value, which induces autocorrelation.
- Since the acceptance probability depends only on the density through ratios, normalizing factors of p and q cancel out.

Symmetric proposals and random walk

If the proposal is symmetric, the ratio of proposal densities is

$$q(\boldsymbol{\theta}_{t-1} \mid \boldsymbol{\theta}_t^*) / q(\boldsymbol{\theta}_t^* \mid \boldsymbol{\theta}_{t-1}) = 1.$$

Common examples include random walk proposals

$$\boldsymbol{\theta}_t^* \leftarrow \boldsymbol{\theta}_{t-1} + \tau Z,$$

where $Z \sim \text{Gauss}(0, 1)$.

Independent proposals

- If we pick instead a global proposal, we must ensure that q samples in far regions (recall rejection sampling), otherwise ...
- Good proposals include heavy tailed distribution such as Student- t with small degrees of freedom,
- centered at the maximum a posteriori $\hat{\boldsymbol{\theta}}$ and
- with scale matrix proportional to $-\mathbf{H}^{-1}(\boldsymbol{\theta}_t^*)$, where $\mathbf{H}(\cdot)$ is the Hessian of the (unnormalized) log posterior.

Upworthy data example

We model the Poisson rates for headlines with questions or not. Our model is

$$Y_i \sim \text{Poisson}(n_i \lambda_i), \quad (i = 1, 2)$$

$$\lambda_1 = \exp(\beta + \kappa)$$

$$\lambda_2 = \exp(\beta)$$

$$\beta \sim \text{Gauss}(\log 0.01, 1.5)$$

$$\kappa \sim \text{Gauss}(0, 1)$$

Implementation details: data and containers

In regression models, scale inputs if possible.

```
1 # Load data
2 data(upworthy_question, package = "hecbayes")
3 # Compute sufficient statistics
4 data <- upworthy_question |>
5   dplyr::group_by(question) |>
6   dplyr::summarize(ntot = sum(impressions),
7                     y = sum(clicks))
8 # Create containers for MCMC
9 niter <- 1e4L
10 chain <- matrix(0, nrow = niter, ncol = 2L)
11 colnames(chain) <- c("beta", "kappa")
```

Implementation details: log posterior function

Perform all calculations on the log scale to avoid numerical overflow!

```
1 # Code log posterior as sum of log likelihood and log prior
2 loglik <- function(par, counts = data$y, offset = data$ntot, ...){
3   lambda <- exp(c(par[1] + log(offset[1]), par[1] + par[2] + log(offset[2])))
4   sum(dpois(x = counts, lambda = lambda, log = TRUE))
5 }
6 # Note common signature of function
7 logprior <- function(par, ...){
8   dnorm(x = par[1], mean = log(0.01), sd = 1.5, log = TRUE) +
9   dnorm(x = par[2], log = TRUE)
10 }
11 logpost <- function(par, ...){
12   loglik(par, ...) + logprior(par, ...)
13 }
```

Implementation details: proposals

Use good starting values for your Markov chains, such as maximum a posteriori.

```
1 # Compute maximum a posteriori (MAP)
2 map <- optim(
3   par = c(-4, 0.07),
4   fn = logpost,
5   control = list(fnscale = -1),
6   offset = data$ntot,
7   counts = data$,
8   hessian = TRUE)
9 # Use MAP as starting value
10 cur <- map$par
11 # Compute logpost_cur - we can keep track of this to reduce calculations
12 logpost_cur <- logpost(cur)
13 # Proposal covariance
14 cov_map <- -2*solve(map$hessian)
15 chol <- chol(cov_map)
```

Implementation details: Metropolis–Hastings algorithm

Use seed for reproducibility, do not compute posterior twice, compute log of acceptance ratio.

```
1 set.seed(80601)
2 naccept <- 0L
3 for(i in seq_len(niter)){
4   # Multivariate normal proposal - symmetric random walk
5   prop <- c(rnorm(n = 2) %*% chol + cur)
6   logpost_prop <- logpost(prop)
7   logR <- logpost_prop - logpost_cur
8   if(logR > -rexp(1)){
9     cur <- prop
10    logpost_cur <- logpost_prop
11    naccept <- naccept + 1L
12  }
13  chain[i,] <- cur
14 }
```

Implementation details: analysis of output

```
1 # Posterior summaries
2 summary(coda::as.mcmc(chain))
```

1. Empirical mean and standard deviation for each variable, plus standard error of the mean:

	Mean	SD	Naive SE	Time-series SE
beta	-4.51268	0.001697	1.697e-05	6.176e-05
kappa	0.07075	0.002033	2.033e-05	9.741e-05

2. Quantiles for each variable:

	2.5%	25%	50%	75%	97.5%
beta	-4.51591	-4.51385	-4.51273	-4.51154	-4.50929
kappa	0.06673	0.06933	0.07077	0.07212	0.07463

References

- Botev, Z., & L'Écuyer, P. (2017). Simulation from the normal distribution truncated to an interval in the tail. *Proceedings of the 10th EAI International Conference on Performance Evaluation Methodologies and Tools on 10th EAI International Conference on Performance Evaluation Methodologies and Tools*, 23–29. <https://doi.org/10.4108/eai.25-10-2016.2266879>
- Devroye, L. (1986). *Non-uniform random variate generation*. Springer.
<http://www.nrbook.com/devroye/>
- Geyer, C. J. (2011). Introduction to Markov chain Monte Carlo. In S. Brooks, A. Gelman, G. Jones, & X. L. Meng (Eds.), *Handbook of Markov chain Monte Carlo* (pp. 3–48). CRC Press.
<https://doi.org/10.1201/b10905>
- Heidelberger, P., & Welch, P. D. (1981). A spectral method for confidence interval generation and run length control in simulations. *Communications of the ACM*, 24(4), 233–245.
<https://doi.org/10.1145/358598.358630>
- Percival, D. B., & Walden, A. T. (2020). *Spectral analysis for univariate time series*. Cambridge University Press. <https://doi.org/10.1017/9781139235723>